MT – Exercise 02

Annalena Alber (24-740-318, a.alber)

1 Training a recurrent neural network language model

Small changes were made to the given scripts to allow for efficient training on a Mac M4 Pro; i.a. changing the device to "mps" and adding the flag `--accel`. The changes can be found in the README.md in my repository.

Present your chosen data, does it have any special attributes that you expect to have an influence on the text generation of the model?

I worked with 26 pseudo-randomly selected episode scripts of the TV show *Friends*. These are structured in a way to show who is speaking, containing almost exclusively dialogue, and stage directions in parentheses. A typical line would look as follows:

> *Rachel: What the hell is that?!! (to Monica) What the hell is that? Is that you? (Monica nods her head no, and Rachel realizes what is making that sound.) Ohhhhhhh! (storms over to Chandler and Joey's with Monica in trail.)*

The start of each episode is its title and the name of the writers:

> *The One With the Monkey*
> *Written by: Adam Chase & Ira Ungerleider*

Each scene starts off with a scene description in square brackets:

> *[Scene: Central Perk, Phoebe and Chandler are watching Monica and Don talk.]*

I expect the character names to show up frequently, as well as the general structures, as described above. The rather short sentences stand out, which makes me expect similarly short sentences in the generated texts.

Take a look at the sample generation, what are your impressions?

> *half) Phoebe : Bye , wait , its gotten him , because Joey didn 't fluid used out who Loreo . <eos>Your new man , she 's small time . <eos>Chandler : Wow , God , the line in his Mockolate . <eos>[Scene : Monica and Rachel 's , Joey is guilt Chandler ' s School Chandler walks to take up except the umbrella . <eos>Joey : It ' s Ray ! <eos>Ross : That is amazing , I ' m afraid ! <eos>Phoebe : I figured what did you know I*

The text itself is only partially sensible, the end of sentence tokens are always in the right locations, and the script structure exists. Given the data set was very small, this nonsense makes sense, and one thing to note is the attempted use of the *[Scene : xxx]* structure, where the opening but no

closing bracket was generated. This can also be seen as an artifact of the data itself, as there are entries where the closing bracket is missing already in the original data.

2 Parameter tuning: Experimenting with dropout

Train at least 5 language models with varying dropout settings (including a model without any dropout). Train on the data you prepared in Task 1.

For automatic batch training use the following script. The dropout was selected to be 0.0, 0.4, 0.5, 0.6, 0.7. The embedding size and number of hidden layers was selected to be 256 each.

Create for each of the three different perplexities a table with the values of each of the different models per epoch/end of training.

Three tables have been created and can be found in the appendix: the training perplexity (Table 2), the validation perplexity (Table 3), and the testing perplexity (Table 1).

Create a line chart each for the training and the validation perplexity to visualize the results.

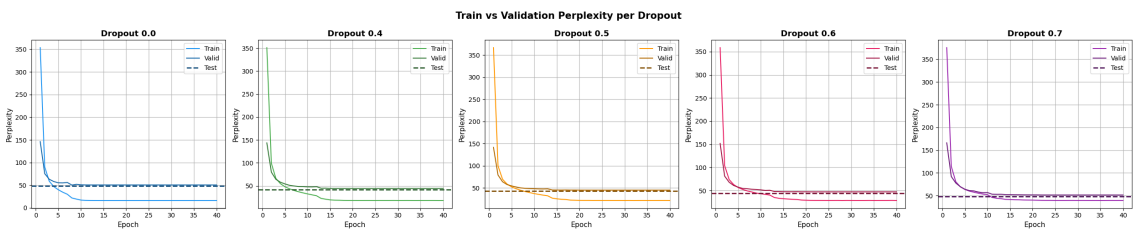

Figure 1: Train vs Validation Perplexity per Dropout

Can you see a connection between the training, validation and test perplexity? Based on your results, which dropout setting do you think is the best and why?

When inspecting directly the correlation between each of the training perplexities and the validation perplexities, there seems to be a connection between the speed at which the training perplexity is lowered and the dropout (see Figure 2). Further, the lower the dropout the faster the plateau is reached (see Figure 3). Generally, the higher the dropout the smaller the delta between the training and validation perplexity, with the actual testing perplexity each time every close to the validation perplexity (see Figure 1).

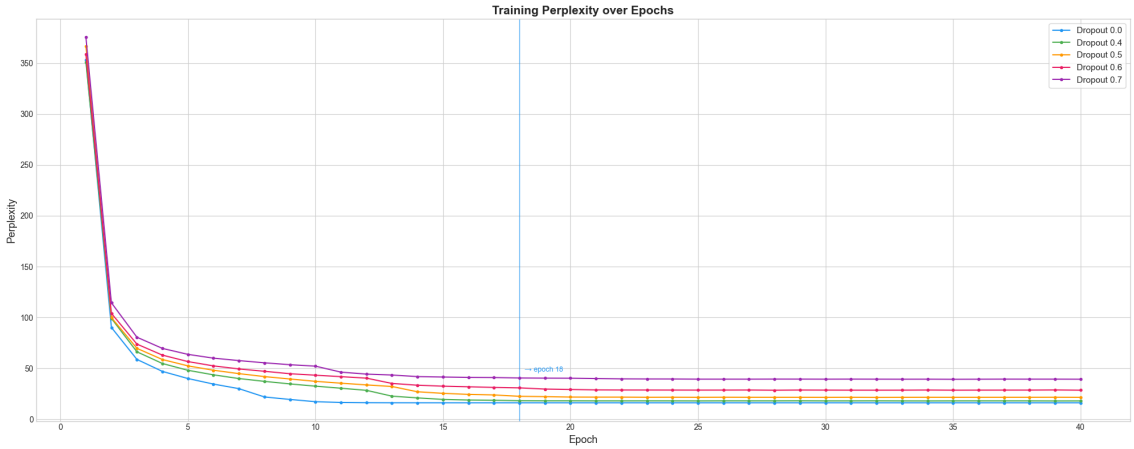

Figure 2: Train Perplexity per Dropout; Vertical Line for Plateau

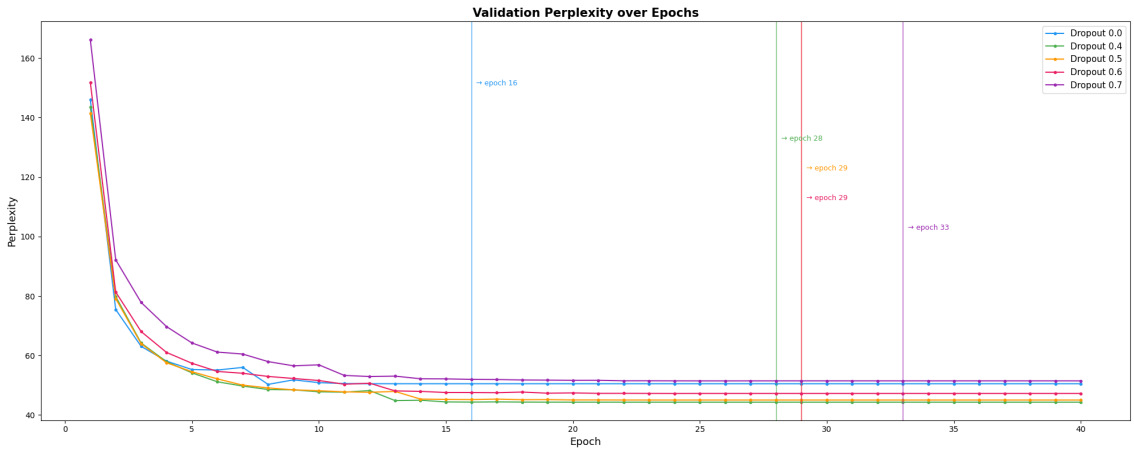

Figure 3: Validation Perplexity per Dropout; Vertical Line for Plateau

Sample some text from the model that obtains the lowest test perplexity. What do you think of its quality? Does it resemble the original training data?

> *half years , has a coupla pin . <eos>Ross : Oh hey ! ¡eos>Chandler : Okay , who Loreo , books . <eos>Oh , she 's small time . <eos>Chandler : Wow , God , the doctor won 't . <eos>Get your ball ! <eos>[Flashback around to Joey area .] <eos>Carol : Oh , no , give me the help I 'm not very continuing . <eos>(Does so .) <eos>CHANDLER : In a couple . <eos>Chandler : Yeah , I figured what ten are for us*

The generated text from the 0.4 dropout model looks very similar to the training data, at least in structure and in the inclusion of the character names. There are many grammatical mistakes in the text, likely due to the structure of the data itself, many sentences seem ungrammatical out of context already in the cleaned data.

Sample some text with the highest test perplexity. Can you see a difference to the lowest scoring one?

> *half) Phoebe : Bye , wait , it 's so fine . <eos>Wow . <eos>Will : Well , some books . <eos>Oh , she 's still ready . <eos>Chandler : Wow , God , the line in us for that , so , I remember it ! <eos>Phoebe : Um , I had a School . <eos>Monica : Yeah , you know I 'm not very playing . <eos>(To Rachel) : That is amazing , I 'm . <eos>Chandler : Yeah , I figured what did you know ,*

Text generated with the worst performing model (dropout 0.7) is still following the general structure of a script. The names of the characters are very present, likely due to the frequency in the original data; all in all there are no large structural differences between the versions.
This lack of significant difference likely stems from the simplicity of the data itself. Many sentences are very short and they follow a very strict structure. Ungrammatical sentences

A Perplexity Tables

dropout	**0.0**	**0.4**	**0.5**	**0.6**	**0.7**
test_ppl					
	47.9860	42.0308	42.5432	44.5721	48.2970

Table 1: Test perplexity

dropout	0.0	0.4	0.5	0.6	0.7
epoch of train_ppl					
1	353.3375	351.1072	366.6375	358.6176	375.5144
2	90.1919	99.0837	99.9180	104.0190	114.4443
3	58.7729	66.2494	69.6528	73.8308	80.5960
4	46.9627	54.7335	58.6505	62.9729	69.5872
5	39.9611	48.0492	52.4149	56.6335	63.7207
6	34.5104	43.5981	48.1050	52.3873	60.0055
7	30.0023	39.9405	44.7797	49.3974	57.5983
8	21.8417	37.1728	41.8683	47.0653	55.4279
9	19.3817	34.6628	39.5007	44.6851	53.5410
10	17.1867	32.4483	37.1989	43.2308	52.1368
11	16.4263	30.4197	35.3711	41.8078	46.2412
12	16.1833	28.3502	33.6774	40.3370	44.3208
13	16.1150	22.6271	32.1392	35.1308	43.4012
14	16.0977	20.9189	26.9261	33.3891	41.8637
15	16.0934	19.4111	25.3992	32.4513	41.4503
16	16.0923	18.8170	24.3756	31.8246	41.0674
17	16.0921	18.5777	23.8098	31.2276	40.9285
18	16.0920	18.1264	22.4786	30.7617	40.5054
19	16.0920	18.0319	22.1541	29.6057	40.3619
20	16.0920	18.0401	21.7492	29.1854	40.2840
21	16.0920	17.9809	21.6801	28.7807	39.9359
22	16.0920	17.9809	21.6820	28.6926	39.6247
23	16.0920	17.9987	21.5301	28.6708	39.5251
24	16.0920	17.9853	21.5714	28.6276	39.5038
25	16.0920	17.9386	21.4989	28.5893	39.3583
26	16.0920	18.0050	21.5714	28.6121	39.3556
27	16.0920	17.9923	21.5201	28.6763	39.3519
28	16.0920	17.9921	21.5034	28.4813	39.4215
29	16.0920	18.0502	21.5068	28.6609	39.4059
30	16.0920	17.9583	21.4641	28.5934	39.3428
31	16.0920	17.9709	21.5254	28.5328	39.4021
32	16.0920	18.0197	21.4322	28.5167	39.3457
33	16.0920	17.9507	21.4836	28.5408	39.3007
34	16.0920	17.9213	21.5202	28.6646	39.3350
35	16.0920	17.9724	21.4901	28.5130	39.2226
36	16.0920	17.9864	21.5034	28.5703	39.3094
37	16.0920	17.9874	21.5107	28.5923	39.3855
38	16.0920	17.9795	21.5180	28.6051	39.3626
39	16.0920	17.9066	21.5543	28.7371	39.3682
40	16.0920	17.9418	21.4709	28.5030	39.2978

Table 2: Training perplexity

dropout	0.0	0.4	0.5	0.6	0.7
epoch of valid_ppl					
1	146.0436	143.5860	141.5030	151.8568	166.1525
2	75.4451	79.8427	79.1240	81.3377	92.1667
3	63.0860	64.1958	63.9771	68.0532	77.8525
4	58.1050	57.9182	57.5716	61.0381	69.7256
5	55.2906	54.1774	54.5436	57.3966	64.2281
6	55.1096	51.1675	52.1452	54.6478	61.1553
7	56.0022	49.7332	50.0489	54.0248	60.4900
8	50.2942	48.5636	49.1042	52.9654	57.9525
9	51.8101	48.4626	48.4621	52.2733	56.5324
10	50.8706	47.8000	48.1508	51.5960	56.8628
11	50.5801	47.6678	47.7822	50.3415	53.3047
12	50.5206	48.2217	47.6383	50.6785	52.9412
13	50.5136	44.8428	47.9197	48.1041	53.0670
14	50.5123	44.9695	45.3517	47.9404	52.1819
15	50.5120	44.3943	45.2515	47.5683	52.1411
16	50.5119	44.3380	45.1924	47.5568	51.9815
17	50.5119	44.4076	45.3469	47.4940	51.9308
18	50.5119	44.3427	45.1492	47.7156	51.7756
19	50.5119	44.3205	45.1820	47.3312	51.7277
20	50.5119	44.3138	45.0561	47.4265	51.6591
21	50.5119	44.3162	45.0612	47.2806	51.6709
22	50.5119	44.3140	45.0381	47.2825	51.4987
23	50.5119	44.3130	45.0328	47.2620	51.5034
24	50.5119	44.3120	45.0285	47.2402	51.4801
25	50.5119	44.3117	45.0318	47.2540	51.4707
26	50.5119	44.3115	45.0316	47.2516	51.4652
27	50.5119	44.3120	45.0317	47.2514	51.4756
28	50.5119	44.3119	45.0318	47.2513	51.4766
29	50.5119	44.3119	45.0319	47.2512	51.4773
30	50.5119	44.3119	45.0319	47.2512	51.4772
31	50.5119	44.3119	45.0319	47.2512	51.4771
32	50.5119	44.3119	45.0319	47.2512	51.4770
33	50.5119	44.3119	45.0319	47.2512	51.4771
34	50.5119	44.3119	45.0319	47.2512	51.4771
35	50.5119	44.3119	45.0319	47.2512	51.4771
36	50.5119	44.3119	45.0319	47.2512	51.4771
37	50.5119	44.3119	45.0319	47.2512	51.4771
38	50.5119	44.3119	45.0319	47.2512	51.4771
39	50.5119	44.3119	45.0319	47.2512	51.4771
40	50.5119	44.3119	45.0319	47.2512	51.4771

Table 3: Validation perplexity